LAB 3

LAB 3 — BUILD AND USE THE SHARED DATASET

Turn the files collected in Lab 2 into point queries, combine sources, then test representations.

PRESS → OR SPACE. →

© Marc Rußwurm
Licensed under **Creative Commons Attribution–NonCommercial 4.0 International (CC BY-NC 4.0)**.
You may share and adapt these slides for teaching, research, and other non-commercial educational purposes with attribution.
Commercial training, paid workshops, consulting seminars, or incorporation into commercial course products require prior permission.

UNIVERSITÄT BONN

Lab 3 Workflow

1. Fork the supplied repository once per group and clone your fork.
2. Connect your Lab 2 file to a download script and a PyTorch dataset.
3. Query named values at coordinates and handle missing or outside-coverage results.
4. Integrate your source into the combined dataset and open a pull request.
5. Use the combined data to compare feature representations and models.

Inputs from Lab 2: the uploaded GeoTIFF, GeoJSON, or shapefile ZIP and your group's two explanatory slides.

Collaborating with GitHub

- **Fork** the instructor’s repository once per group.
- **Clone** the group fork into your working environment.
- Develop together; **commit** and **push** working changes.
- Open a **pull request** to the central repository.
- Review, revise, and merge with the instructor.

Starting repository: *[LINK TO BE PROVIDED]*

Make Your Source Queryable

1. Read your group’s two Lab 2 slides and download the uploaded subset.
2. Adapt the supplied download script to retrieve the file and cache it in `data/`.
3. Adapt the land-cover or building example into a PyTorch dataset for your source.
4. Return named variables for a requested latitude and longitude.
5. Transform coordinates as needed; distinguish missing values from valid zeros.
6. Check known locations against the original map or source data.

The file is downloaded when needed, then queried locally. A point query should not download it again.

Integrate the Group Contribution

1. Agree on variable names and the point-query interface.
2. Connect your dataset to the combined dataset script.
3. Confirm the combined query returns your source alongside the supplied examples.
4. Run the repository checks and inspect plausible results at known points.
5. Push the change, open a pull request, and respond to review.

Technical deliverable: a reviewed contribution that makes your Lab 2 source available through the shared query.

Combined Point Query

```
dataset.query(latitude=..., longitude=...)
```

```
{
    "land_cover":     ...,
    "building":       ...,
    "your_modality":  ...
}
```

Each source may have its own resolution, CRS, and observation date.

Keep variable meanings and missing-data conventions from the Lab 2 slides.

Use the repository's actual query interface; this example is illustrative.

The Task

- Classify land cover from the dataset assembled in this lab
- Work with point-based samples from raster/geospatial layers
- Use one dataset consistently across all experiments
- Evaluate predictions on held-out data

For the model comparison, use Python in **Google Colab**, **VS Code**, or local Jupyter. CPU is enough.

Experimental Questions

Q1
Does feature engineering help?

Q2
Does complementary information help?

Q3
When do we need a nonlinear model?

Treat this as an experiment: hold conditions fixed and change one conceptual factor at a time.

Build the Dataset Once

- Choose/extract labeled sample locations
- Build one table: **rows = locations**, **columns = features**
- Target column: land-cover class
- Create one fixed train / validation / test split

Important: Do the split once and reuse it for every experiment.

Baseline: Logistic Regression

- Use `LogisticRegression` as the linear baseline
- Input: feature vector per sample
- Learns a linear decision boundary
- Fast, simple, and interpretable reference model

Key question: How good is your representation if linear separation is all you allow?

Change the Representation (Model Fixed)

Retrain the same `LogisticRegression` while changing features:

1. Small raw feature set (e.g., Red + NIR)
2. Raw features + engineered feature (e.g., NDVI)
3. Several Sentinel-2 bands
4. Bands + complementary variables from the combined dataset (if available)

Prompt: What changes when the model stays fixed but the representation changes?

Compare with Random Forest

- Run `RandomForestClassifier` on the same representations
- Keep the exact same train/validation/test split
- Compare against logistic regression fairly

Logistic Regression
Linear decision boundary

Random Forest
Nonlinear decision boundaries

Ablation Table

Representation	Logistic Regression	Random Forest
Raw baseline	…	…
+ NDVI	…	…
+ additional bands	…	…
+ complementary features	…	…

Use accuracy (and optionally macro-F1). Focus on interpreting changes, not maximizing one score.

Interpret the Experiment

- Which representation change helped most?
- When did logistic regression approach random-forest performance?
- What does the model gap suggest about feature-space geometry?

Caution: More dimensions are not automatically better. Useful, complementary information matters.

Takeaway → Lecture 4

1. Feature engineering changes the representation.
2. Complementary information can make classification easier.
3. A strong representation can make simple linear models surprisingly powerful.

NEXT

In Lab 3 you designed representations manually.
In Lecture 4: what if the model learns the representation itself?